Zadanie nr 1 - Trening neuronu liniowego regułą delta z użyciem wielu wzorców treningowych

Inteligentna Analiza Danych

Jędrzej Małaczyński 264351 Jędrzej Kostyk 264339

25 października 2026

1 Cel zadania

Celem zadania było zaimplementowanie treningu pojedynczego neuronu liniowego regułą delta dla zbioru złożonego z wielu wzorców treningowych oraz zbadanie, jak przebieg i wynik treningu zależą od relacji między liczbą wag neuronu N a liczbą wzorców M. Dla każdego z przypadków $N < M$, $N = M$ i $N > M$ sprawdzono, czy wagi końcowe zależą od wag początkowych oraz czy możliwe jest bezbłędne nauczenie neuronu całego zbioru treningowego. Obserwacje zinterpretowano, traktując trening jako rozwiązywanie układu M równań liniowych z N niewiadomymi.

2 Wstęp teoretyczny

Neuron liniowy o wektorze wag $\mathbf{w} \in \mathbb{R}^N$ odpowiada na wzorzec $\mathbf{x}^\mu$ wartością $y^\mu = \mathbf{w}^T \mathbf{x}^\mu$. Trening polega na takim doborze wag, aby odpowiedzi były jak najbliższe wartościom żądanym z^μ, tj. na minimalizacji błędu średniokwadratowego (MSE) na zbiorze treningowym [2, 1]. Zastosowano regułę delta (Widrowa–Hoffa, LMS) [5] w wariancie *on-line* odpowiadającym Algorytmowi 3.2 z wykładu [2]: w każdej epoce wzorce są prezentowane kolejno, a wagi są poprawiane po każdym z nich

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left(z^\mu - \mathbf{w}^T \mathbf{x}^\mu\right) \mathbf{x}^\mu, \qquad \mu = 1, \ldots, M. \tag{1}$$

Po zebraniu wzorców w wiersze macierzy $X \in \mathbb{R}^{M \times N}$, a wartości żądanych w wektor $\mathbf{z} \in \mathbb{R}^M$, bezbłędny trening oznacza rozwiązanie układu $X\mathbf{w} = \mathbf{z}$. Zgodnie z twierdzeniem Kroneckera–Capellego dla danych losowych (rząd X pełny z prawdopodobieństwem 1) układ jest: dla $N < M$ – na ogół sprzeczny, dla $N = M$ – oznaczony, a dla $N > M$ – nieoznaczony (rozwiązania tworzą podprzestrzeń afiniczną wymiaru $N - M$) [3]. Jako wynik odniesienia użyto $X^+\mathbf{z}$, gdzie X^+ jest pseudoodwrotnością Moore'a–Penrose'a [4]: dla $N < M$ jest to rozwiązanie najmniejszych kwadratów (LS), dla $N = M$ – $X^{-1}\mathbf{z}$, a dla $N > M$ – rozwiązanie o minimalnej normie.

Kluczowa własność reguły (1): każda poprawka jest wielokrotnością wektora wzorca $\mathbf{x}^\mu$, więc leży w przestrzeni rozpiętej przez wzorce. Składowa wag początkowych $\mathbf{w}_0$ leżąca w jądrze $\ker X$ (prostopadła do wszystkich wzorców) nigdy nie jest zmieniana. Jeśli układ jest niesprzeczny i algorytm zbiega, to wagi końcowe powinny mieć postać

$$\hat{\mathbf{w}} = X^+\mathbf{z} + \left(I - X^+X\right)\mathbf{w}_0, \tag{2}$$

gdzie $(I - X^+X)$ jest rzutem ortogonalnym na $\ker X$. Hipotezę tę zweryfikowano eksperymentalnie.

3 Eksperymenty i wyniki

Wspólne założenia wszystkich eksperymentów:

- składowe wzorców i wartości żądane losowane z rozkładu $U(-1, 1)$ przy stałym ziarnie generatora (42) – **ten sam zbiór treningowy** we wszystkich wykonaniach,
- wagi początkowe z $U(-1, 1)$, **inne w każdym wykonaniu** (ziarna 100–104),
- współczynnik uczenia $\eta = 0{,}01$, liczba epok $K = 10\,000$, 5 wykonań na przypadek,
- po każdej epoce liczono MSE na całym zbiorze treningowym; po treningu – maksymalny błąd bezwzględny $\max_\mu |z^\mu - y^\mu|$ oraz odległości euklidesowe między wagami,
- implementacja w Pythonie (NumPy), X^+ liczone funkcją `pinv` z modułu `numpy.linalg`.

3.1 Eksperyment nr 1 – mniej wag niż wzorców ($N = 2$, $M = 6$)

3.1.1 Założenia

Neuron o $N = 2$ wagach trenowano na $M = 6$ wzorcach, co odpowiada układowi 6 równań z 2 niewiadomymi. Macierz X ma rząd 2 i współczynnik uwarunkowania $\mathrm{cond}(X) = 1{,}21$, więc rozwiązanie LS jest jednoznaczne.

3.1.2 Przebieg

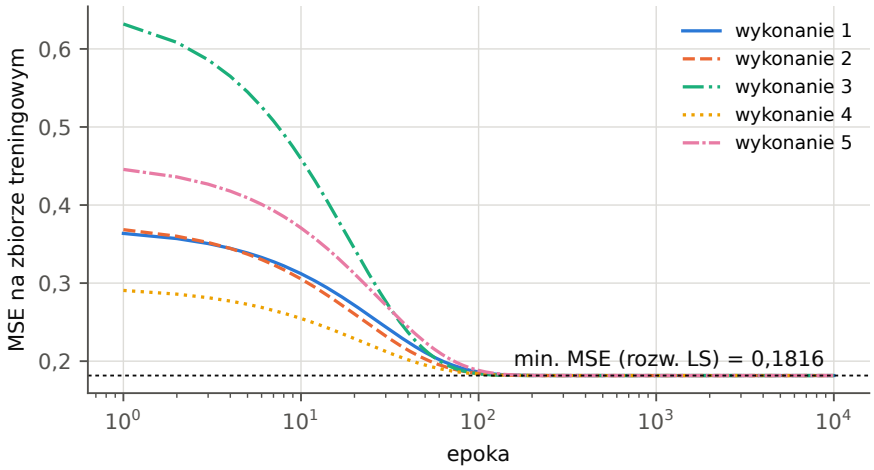
Algorytm uruchomiono 5 razy z różnych wag początkowych. Wagi końcowe porównano między sobą oraz z rozwiązaniem LS $X^+\mathbf{z}$. Dodatkowo, aby wyjaśnić niewielką różnicę między wynikiem reguły delta a rozwiązaniem LS, powtórzono trening dla różnych wartości η ($K = 100\,000$ epok).

3.1.3 Rezultat

Wyniki przedstawiono w Tab. 1, a przebieg błędu na Rys. 1.
Mimo różnych punktów startowych wszystkie wykonania zakończyły się **identycznymi wagami** – maksymalna odległość między nimi wynosi $3{,}8 \cdot 10^{-15}$, czyli błąd zaokrągleń arytmetyki zmiennoprzecinkowej. Błąd nie spadł jednak do zera: po ok. 100 epokach ustalił się na poziomie MSE = 0,1816,

Tabela 1: Wagi początkowe i końcowe dla $N = 2$, $M = 6$

Wyk.	$w_1^{(0)}$	$w_2^{(0)}$	w_1	w_2	**MSE**	$\max\lvert z - y\rvert$
1	0,6700	0,1931	0,1688	−0,4164	0,1816	0,6888
2	0,8871	−0,2812	0,1688	−0,4164	0,1816	0,6888
3	−0,6800	0,1719	0,1688	−0,4164	0,1816	0,6888
4	−0,3746	−0,5514	0,1688	−0,4164	0,1816	0,6888
5	0,6771	0,3843	0,1688	−0,4164	0,1816	0,6888
X^+z	–	–	0,1710	−0,4140	0,1816	–



Rysunek 1: Przebieg MSE w kolejnych epokach dla $N = 2$, $M = 6$ (oś epok logarytmiczna)

równym minimum osiągalnemu przez rozwiązanie LS (Rys. 1), a błąd dla najgorszego wzorca to 0,689. Neuron **nie został nauczony bezbłędnie** – nie istnieje wektor wag spełniający jednocześnie wszystkie 6 równań, więc algorytm znajduje kompromis minimalizujący sumę kwadratów błędów.

Wagi końcowe różnią się od $X^+\mathbf{z}$ o $3{,}2\cdot10^{-3}$. Tab. 2 pokazuje, że różnica ta jest proporcjonalna do η (iloraz w ostatniej kolumnie jest stały), a nadwyżka MSE ponad minimum – do η^2. Wynika to z wariantu *on-line*: gdy układ jest sprzeczny, każdy wzorzec „ciągnie” wagi w stronę własnego rozwiązania, więc przy stałym η wagi kończą epokę w punkcie przesuniętym względem minimum LS. Dla $\eta \to 0$ wynik dąży do rozwiązania LS.

Tabela 2: Wpływ η na wynik dla $N = 2$, $M = 6$ ($K = 100\,000$)

$\boldsymbol{\eta}$	$\|\mathbf{w} - X^{+}\mathbf{z}\|$	$\mathrm{MSE} - \mathrm{MSE}_{LS}$	$\|\mathbf{w} - X^{+}\mathbf{z}\|/\eta$
0,1	$3{,}31 \cdot 10^{-2}$	$3{,}36 \cdot 10^{-4}$	0,331
0,03	$9{,}73 \cdot 10^{-3}$	$2{,}91 \cdot 10^{-5}$	0,324
0,01	$3{,}23 \cdot 10^{-3}$	$3{,}19 \cdot 10^{-6}$	0,323
0,003	$9{,}66 \cdot 10^{-4}$	$2{,}86 \cdot 10^{-7}$	0,322
0,001	$3{,}22 \cdot 10^{-4}$	$3{,}18 \cdot 10^{-8}$	0,322

3.2 Eksperyment nr 2 – liczba wag równa liczbie wzorców ($N = M = 4$)

3.2.1 Założenia

Układ 4 równań z 4 niewiadomymi. Macierz X jest nieosobliwa (rząd 4), jej wartości osobliwe to 1,672; 1,169; 1,047; 0,246, a współczynnik uwarunkowania $\mathrm{cond}(X) = 6{,}79$. Układ ma więc dokładnie jedno rozwiązanie $X^{-1}\mathbf{z}$.

3.2.2 Przebieg

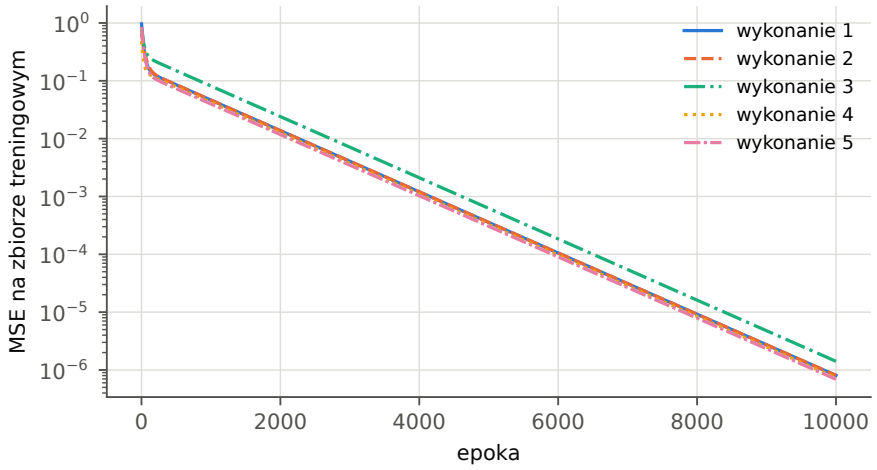
Jak w eksperymencie nr 1: 5 wykonań z różnych wag początkowych, porównanie wag końcowych między sobą i z $X^{-1}\mathbf{z}$. Ze względu na wolniejszą zbieżność dodatkowo powtórzono trening dla $K = 30\,000$ epok.

3.2.3 Rezultat

Tabela 3: Wagi końcowe dla $N = M = 4$ ($K = 10\,000$)

Wyk.	w_1	w_2	w_3	w_4	**MSE**	$\|\mathbf{w} - X^{-1}\mathbf{z}\|$
1	1,8519	0,7432	−2,1381	1,8193	$8{,}1 \cdot 10^{-7}$	$7{,}3 \cdot 10^{-3}$
2	1,8519	0,7432	−2,1381	1,8193	$8{,}1 \cdot 10^{-7}$	$7{,}3 \cdot 10^{-3}$
3	1,8506	0,7424	−2,1368	1,8181	$1{,}4 \cdot 10^{-6}$	$9{,}7 \cdot 10^{-3}$
4	1,8521	0,7433	−2,1382	1,8195	$7{,}4 \cdot 10^{-7}$	$7{,}0 \cdot 10^{-3}$
5	1,8522	0,7434	−2,1384	1,8196	$6{,}9 \cdot 10^{-7}$	$6{,}8 \cdot 10^{-3}$
$X^{-1}z$	1,8559	0,7459	−2,1420	1,8232	0	0

Na Rys. 2 błąd maleje wykładniczo (prosta w skali logarytmicznej) i w granicy dąży do zera – **trening bezbłędny jest możliwy**. Po 10 000 epokach MSE wynosi $7 \cdot 10^{-7}$–$1{,}4 \cdot 10^{-6}$, a maksymalny błąd dla pojedynczego wzorca nie przekracza $1{,}6 \cdot 10^{-3}$. Wagi końcowe z różnych wykonań pokrywają się



Rysunek 2: Przebieg MSE w kolejnych epokach dla $N = M = 4$ (oś błędu logarytmiczna)

do trzeciego miejsca po przecinku (maksymalna odległość $2{,}9 \cdot 10^{-3}$, Tab. 3); pozostała różnica wynika wyłącznie z niezakończonej zbieżności. Po 30 000 epokach odległość każdego wykonania od $X^{-1}\mathbf{z}$ spadła do $5{,}0 \cdot 10^{-8}$, więc wszystkie wykonania zbiegają do **tego samego, jedynego rozwiązania**.

Zbieżność jest tu wyraźnie wolniejsza niż w eksperymencie nr 1. Najwolniej korygowana jest składowa wag w kierunku związanym z najmniejszą wartością osobliwą $\sigma_{\min} = 0{,}246$ – im gorzej uwarunkowany układ, tym więcej epok potrzeba.

3.3 Eksperyment nr 3 – więcej wag niż wzorców ($N = 6$, $M = 3$)

3.3.1 Założenia

Układ 3 równań z 6 niewiadomymi. Rząd X wynosi 3, więc $\dim \ker X = N - M = 3$: rozwiązań jest nieskończenie wiele i tworzą trójwymiarową podprzestrzeń afiniczną w $\mathbb{R}^6$.

3.3.2 Przebieg

5 wykonań z różnych wag początkowych. Poza porównaniem wag końcowych między sobą i z rozwiązaniem o minimalnej normie $X^+\mathbf{z}$ sprawdzono, czy wagi końcowe są zgodne z przewidywaniem (2), tzn. obliczono $\|\mathbf{w} - \hat{\mathbf{w}}\|$.

3.3.3 Rezultat

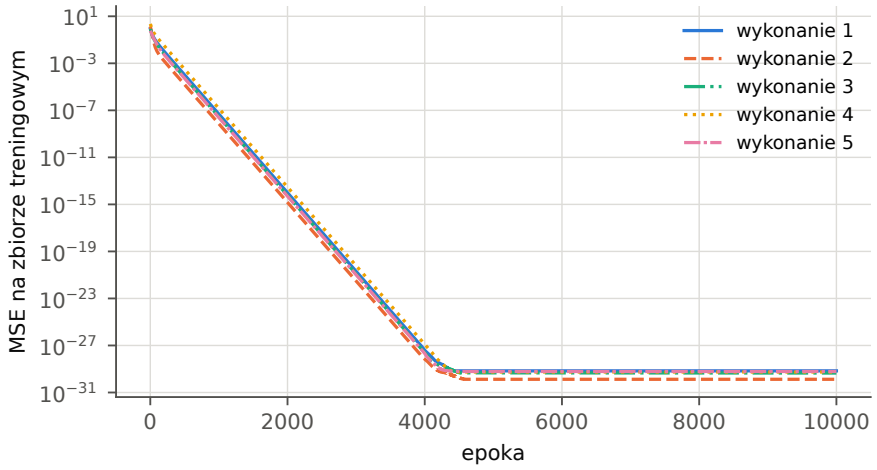
Tabela 4: Wagi końcowe dla $N = 6$, $M = 3$

Wyk.	w_1	w_2	w_3	w_4	w_5	w_6
1	1,0447	0,0579	0,6460	−0,7345	0,3556	0,2157
2	0,9011	0,1433	0,3802	−0,2842	−0,1093	−0,0737
3	0,0905	0,7800	0,7457	−0,5782	−0,1847	0,2570
4	0,9501	0,2997	0,1050	0,5125	−0,3832	−0,4387
5	0,6270	0,0794	0,3630	−0,6584	−0,4106	−0,0129
X^+z	0,5330	0,3997	0,3637	−0,1725	−0,3982	−0,1093

Tabela 5: Błędy i odległości wag końcowych dla $N = 6$, $M = 3$

Wyk.	**MSE**	$\max \lvert z - y \rvert$	$\|\mathbf{w} - X^+\mathbf{z}\|$	$\|\mathbf{w} - \hat{\mathbf{w}}\|$	$\|\mathbf{w}\|$
1	$7{,}0 \cdot 10^{-30}$	$3{,}3 \cdot 10^{-15}$	1,2034	$1{,}5 \cdot 10^{-14}$	1,4915
2	$1{,}3 \cdot 10^{-30}$	$1{,}7 \cdot 10^{-15}$	0,5466	$8{,}7 \cdot 10^{-15}$	1,0370
3	$4{,}5 \cdot 10^{-30}$	$3{,}2 \cdot 10^{-15}$	0,9114	$1{,}3 \cdot 10^{-14}$	1,2678
4	$5{,}7 \cdot 10^{-30}$	$3{,}1 \cdot 10^{-15}$	0,9104	$7{,}7 \cdot 10^{-15}$	1,2670
5	$6{,}2 \cdot 10^{-30}$	$3{,}3 \cdot 10^{-15}$	0,5975	$8{,}1 \cdot 10^{-15}$	1,0646
X^+z	0	0	0	–	0,8812

W każdym wykonaniu **trening był skuteczny**: MSE spadł do poziomu 10^{-30}, czyli do granicy precyzji liczb typu `float64` (Rys. 3, po ok. 4 500



Rysunek 3: Przebieg MSE w kolejnych epokach dla $N = 6$, $M = 3$ (oś błędu logarytmiczna)

epokach), a błąd dla żadnego wzorca nie przekracza $3{,}4 \cdot 10^{-15}$. Jednocześnie **wagi końcowe są w każdym wykonaniu inne** (Tab. 4) – maksymalna odległość między nimi wynosi 1,76. Każdy z tych wektorów jest równie dobrym rozwiązaniem układu.

Tab. 5 potwierdza przewidywanie (2): wagi końcowe różnią się od $\hat{\mathbf{w}}$ jedynie o 10^{-14}, a więc są w pełni wyznaczone przez wagi początkowe. Trening „poprawia" tylko składową wag w przestrzeni rozpiętej przez wzorce, a składowa z $\ker X$ pozostaje taka, jaką wylosowano. Żadne wykonanie nie trafiło w rozwiązanie o minimalnej normie ($\|X^{+}\mathbf{z}\| = 0{,}881$, podczas gdy $\|\mathbf{w}\| \in [1{,}037;\ 1{,}492]$); osiągnęłoby je tylko wykonanie startujące z $\mathbf{w}_0 = \mathbf{0}$.

3.4 Eksperyment nr 4 – interpretacja geometryczna

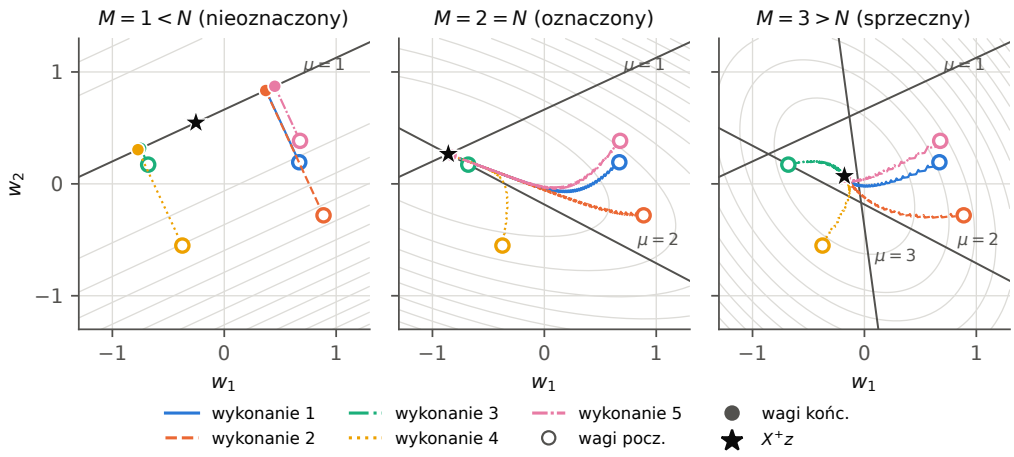
3.4.1 Założenia

Aby zobrazować wyniki eksperymentów 1–3, użyto neuronu o $N = 2$ wagach, dla którego przestrzeń wag jest płaszczyzną. Każde równanie $\mathbf{w}^T\mathbf{x}^\mu = z^\mu$ jest w niej prostą. Wylosowano 3 wzorce (ziarno 18) i trenowano neuron kolejno na pierwszym wzorcu ($M = 1$), dwóch ($M = 2$) i trzech ($M = 3$), za każdym razem z tych samych 5 punktów startowych. Dla czytelności trajektorii przyjęto $\eta = 0{,}1$ i $K = 300$.

3.4.2 Przebieg

Po każdej poprawce wag zapisywano położenie wektora $\mathbf{w}$ i rysowano trajektorie na tle poziomic funkcji MSE.

3.4.3 Rezultat



Rysunek 4: Trajektorie wag w płaszczyźnie (w_1, w_2); szare proste – równania $\mathbf{w}^T\mathbf{x}^\mu = z^\mu$, szare krzywe – poziomice MSE

Rys. 4 pokazuje wszystkie trzy przypadki na raz:

- $M = 1 < N$ – rozwiązania wypełniają całą prostą. Każda trajektoria biegnie prostopadle do niej, więc wagi końcowe są rzutem ortogonalnym wag początkowych na prostą – różne dla każdego wykonania (odpowiednik eksperymentu nr 3).
- $M = 2 = N$ – dwie proste przecinają się w jednym punkcie i wszystkie trajektorie do niego zbiegają (odpowiednik eksperymentu nr 2).
- $M = 3 > N$ – trzy proste nie mają wspólnego punktu (układ sprzeczny), więc zerowy błąd jest nieosiągalny. Wszystkie trajektorie kończą się w tym samym punkcie wewnątrz trójkąta utworzonego przez proste, w pobliżu rozwiązania LS; widoczne drobne zygzaki to efekt poprawek *on-line* od kolejnych wzorców (odpowiednik eksperymentu nr 1).

4 Wnioski

Tabela 6: Zestawienie wyników eksperymentów 1–3

	$N < M$	$N = M$	$N > M$
Układ równań	sprzeczny	oznaczony	nieoznaczony
Końcowe MSE	0,1816 ($= \mathrm{MSE}_{LS}$)	$\to 0$	$\approx 10^{-30}$
Wagi końcowe	identyczne	identyczne	różne
Zależność od $\mathbf{w}_0$	brak	brak	wzór (2)
Skuteczny trening	nie	tak	tak

1. Trening neuronu liniowego regułą delta jest iteracyjnym rozwiązywaniem układu $X\mathbf{w} = \mathbf{z}$, a wszystkie obserwacje (Tab. 6) wynikają z tego, ile rozwiązań ma ten układ.

2. Dla $N < M$ układ jest sprzeczny, więc **bezbłędny trening nie jest możliwy**. Algorytm znajduje jedno, niezależne od wag początkowych rozwiązanie kompromisowe, bliskie rozwiązaniu najmniejszych kwadratów. W wariancie *on-line* ze stałym η jest ono przesunięte względem LS o wartość proporcjonalną do η.

3. Dla $N = M$ (przy $\det X \neq 0$) układ ma jedno rozwiązanie: trening jest skuteczny, a wagi końcowe nie zależą od punktu startowego. Liczba potrzebnych epok rośnie wraz z pogarszaniem się uwarunkowania macierzy wzorców.

4. Dla $N > M$ trening jest zawsze skuteczny, ale wagi końcowe nie są jednoznaczne: reguła delta zmienia wagi tylko w przestrzeni rozpiętej przez wzorce, a składowa wag początkowych z $\ker X$ przechodzi bez zmian do wyniku. Wagi końcowe są dokładnie przewidywane wzorem (2).

5. O niejednoznaczności decyduje właściwie $\dim \ker X = N - \operatorname{rank} X$, a nie samo porównanie N i M: przy liniowo zależnych wzorcach podobny efekt wystąpiłby także dla $N \leq M$.

6. Praktyczny wniosek: neuron o większej liczbie wag niż wzorców dopasowuje zbiór treningowy bez błędu, ale jego wagi są w dużej części losowe, więc zerowy błąd treningowy nie świadczy o poprawnym modelu. Potrzeba wtedy więcej danych lub dodatkowego kryterium wyboru rozwiązania (np. minimalnej normy).

5 Załączniki

Implementacja reguły delta (fragment pliku `neuron_liniowy.py`). Pełny kod eksperymentów i wykresów znajduje się w plikach `neuron_liniowy.py` oraz `wykresy.py`.

```
def delta_training(X, Z, eta=0.01, K=100, w_range=(-1, 1), rng=None):
    """Regula delta on-line: po kazdym wzorcu w <- w + eta*(z - y)*x."""
    rng = rng or np.random.default_rng()
    M, N = X.shape
    w0 = rng.uniform(w_range[0], w_range[1], N)
    w = w0.copy()
    mse = np.empty(K)
    for k in range(K):
        for mu in range(M):
            y = w @ X[mu]
            e = Z[mu] - y
            w = w + eta * e * X[mu]
        mse[k] = np.mean((Z - X @ w) ** 2) # blad na calym zbiorze
    return w0, w, mse
```

Bibliografia

[1] Simon Haykin. *Neural Networks and Learning Machines*. Pearson Prentice Hall, 3rd edition, 2009.

[2] Materiały do wykładu nr 1 – trening neuronu liniowego regułą delta (algorytm 3.2). Kurs Inteligentna Analiza Danych, Politechnika Łódzka.

[3] Stanisław Osowski. *Sieci neuronowe do przetwarzania informacji*. Oficyna Wydawnicza Politechniki Warszawskiej, 2000.

[4] Roger Penrose. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society*, 51(3):406–413, 1955.

[5] Bernard Widrow and Marcian E. Hoff. Adaptive switching circuits. In *IRE WESCON Convention Record*, volume 4, pages 96–104, 1960.